

LAB DAY 1

NAME: AGILAN VIYASH S

REG. NO.: 192511346

COURSE CODE AND NAME: CSA0617 DESIGN AND ANALYSIS OF ALGORITHM

LINEAR SEARCH (ITERATIVE)

PYTHON CODE:

```
# Linear Search - Iterative
```

```
def linear_search(arr, key):
```

```
    for i in range(len(arr)):
```

```
        if arr[i] == key:
```

```
            return i
```

```
    return -1
```

```
# Sample Input
```

```
arr = [10, 25, 30, 45, 50]
```

```
key = 30
```

```
# Perform Linear Search
```

```
result = linear_search(arr, key)
```

```
# Display Output
```

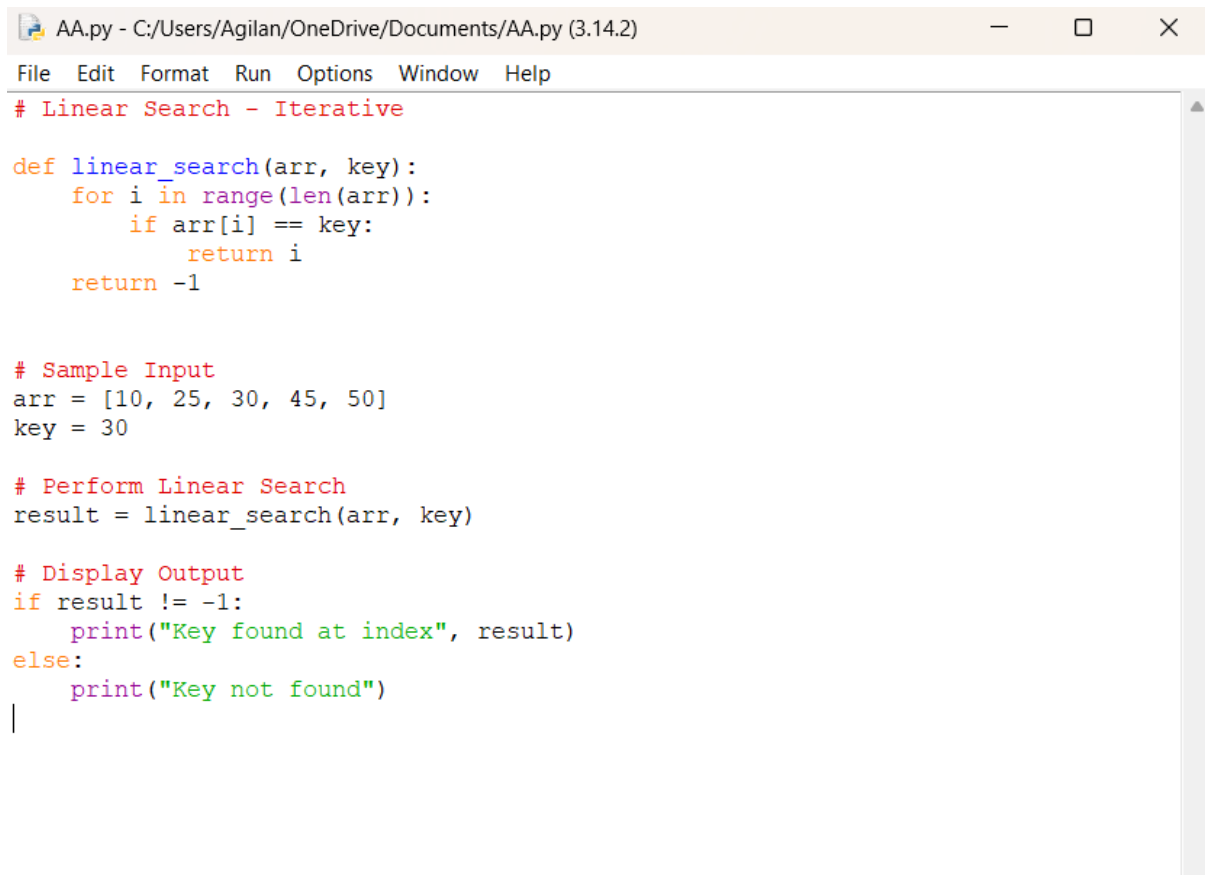
```
if result != -1:
```

```
    print("Key found at index", result)
```

```
else:
```

```
    print("Key not found")
```

EXECUTION:



The screenshot shows a Python IDE window titled "AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)". The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code is as follows:

```
# Linear Search - Iterative

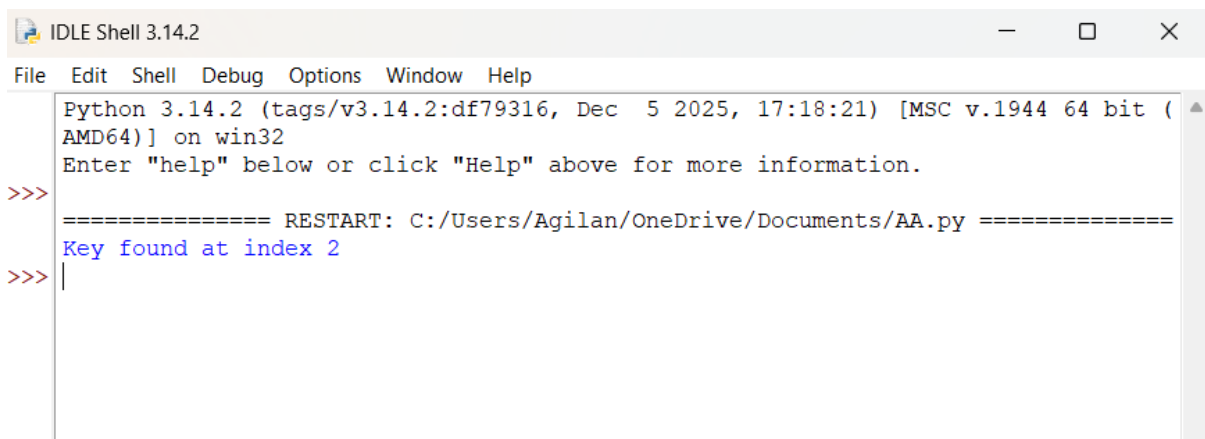
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Sample Input
arr = [10, 25, 30, 45, 50]
key = 30

# Perform Linear Search
result = linear_search(arr, key)

# Display Output
if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

OUTPUT:



The screenshot shows an IDLE Shell window titled "IDLE Shell 3.14.2". The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The output is as follows:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Key found at index 2
>>> |
```

BINARY SEARCH (RECURSIVE)

PYTHON CODE:

Binary Search - Recursive

```
def binary_search(arr, low, high, key):  
    if low <= high:  
        mid = (low + high) // 2  
  
        # Key found  
        if arr[mid] == key:  
            return mid  
  
        # Search in the left half  
        elif key < arr[mid]:  
            return binary_search(arr, low, mid - 1, key)  
  
        # Search in the right half  
        else:  
            return binary_search(arr, mid + 1, high, key)  
  
    return -1
```

```
# Sample Input
```

```
arr = [5, 10, 15, 20, 25]
```

```
key = 20
```

```
# Perform Binary Search
```

```
result = binary_search(arr, 0, len(arr) - 1, key)
```

```
# Display Output
```

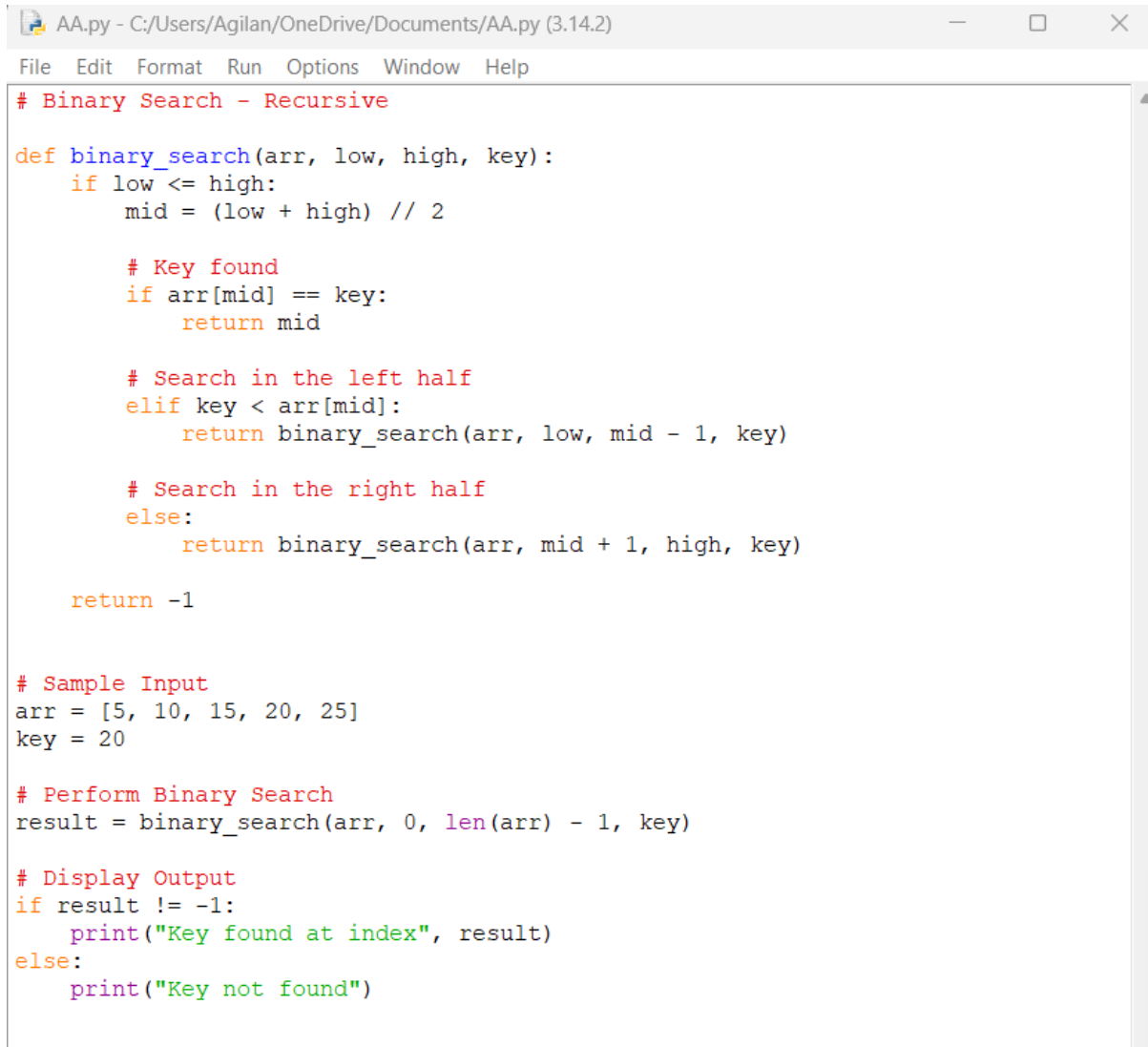
```
if result != -1:
```

```
    print("Key found at index", result)
```

else:

```
print("Key not found")
```

EXECUTION:



The screenshot shows a Python IDE window titled 'AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code is as follows:

```
# Binary Search - Recursive

def binary_search(arr, low, high, key):
    if low <= high:
        mid = (low + high) // 2

        # Key found
        if arr[mid] == key:
            return mid

        # Search in the left half
        elif key < arr[mid]:
            return binary_search(arr, low, mid - 1, key)

        # Search in the right half
        else:
            return binary_search(arr, mid + 1, high, key)

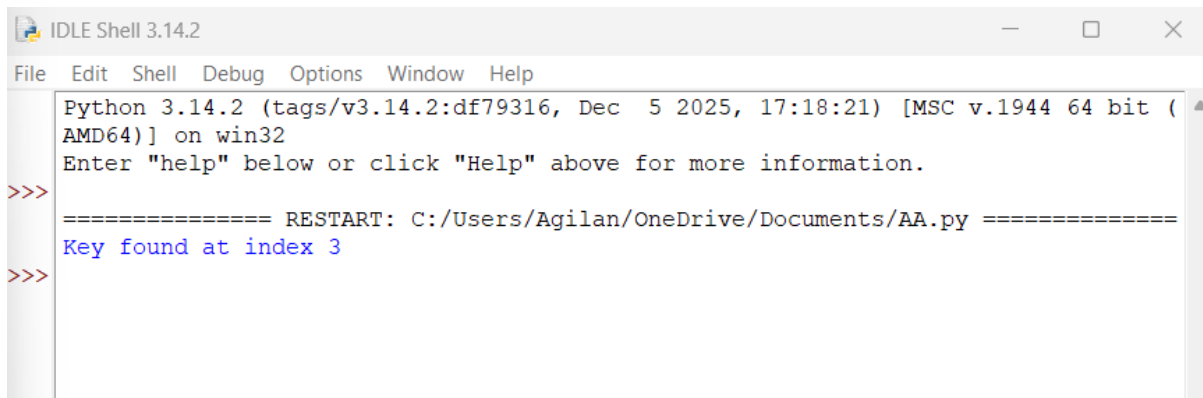
    return -1

# Sample Input
arr = [5, 10, 15, 20, 25]
key = 20

# Perform Binary Search
result = binary_search(arr, 0, len(arr) - 1, key)

# Display Output
if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

OUTPUT:



The screenshot shows an IDLE Shell window titled 'IDLE Shell 3.14.2'. The menu bar includes File, Edit, Shell, Debug, Options, Window, and Help. The output is as follows:

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Key found at index 3
>>>
```

FACTORIAL – ITERATIVE VS RECURSIVE

PYTHON CODE:

Factorial using Iterative Method

```
def factorial_iterative(n):
```

```
    result = 1
```

```
    for i in range(1, n + 1):
```

```
        result *= i
```

```
    return result
```

Factorial using Recursive Method

```
def factorial_recursive(n):
```

```
    if n == 0 or n == 1:
```

```
        return 1
```

```
    else:
```

```
        return n * factorial_recursive(n - 1)
```

Sample Input

```
n = 5
```

Calculate factorial

```
iterative_result = factorial_iterative(n)
```

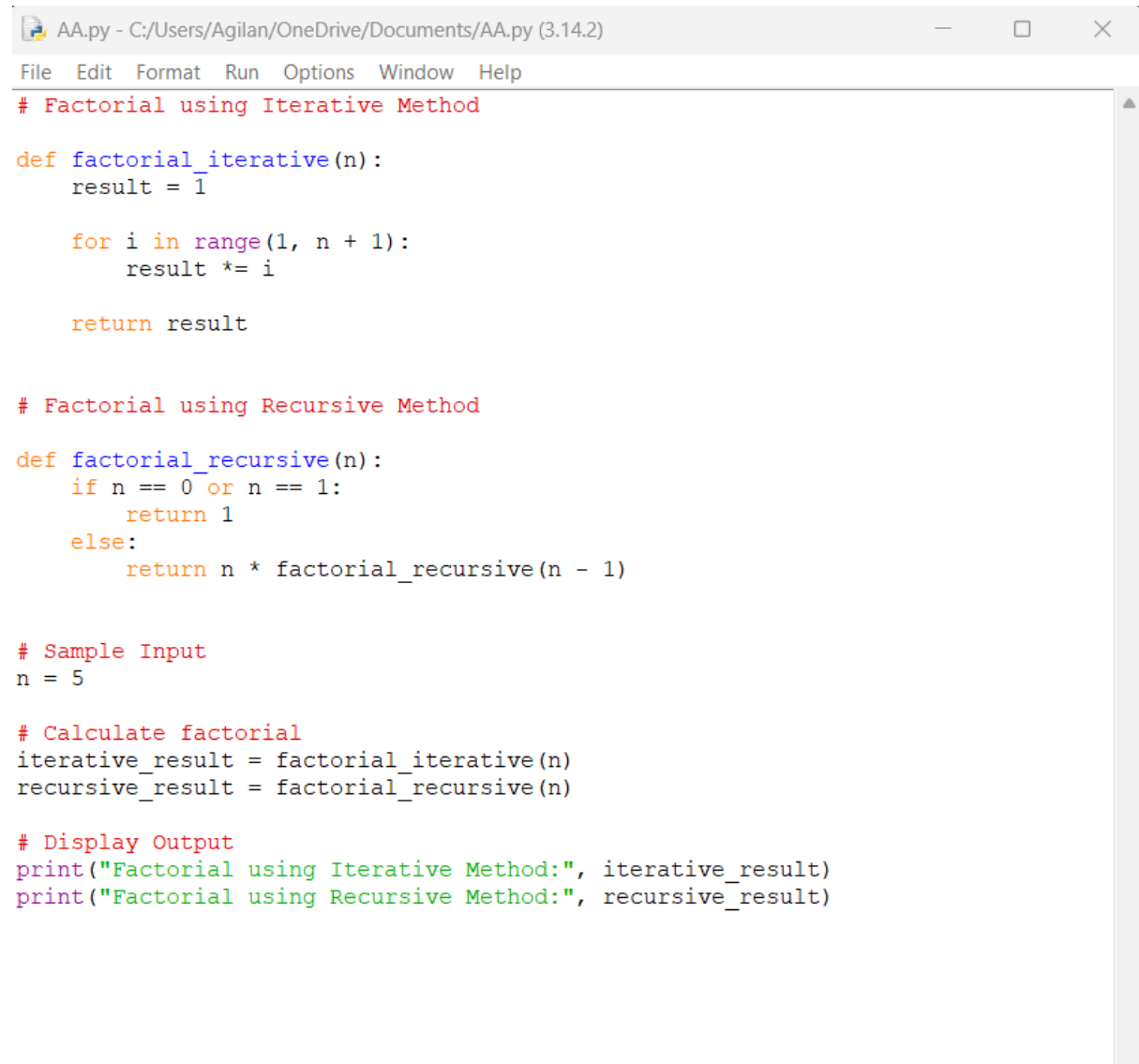
```
recursive_result = factorial_recursive(n)
```

Display Output

```
print("Factorial using Iterative Method:", iterative_result)
```

```
print("Factorial using Recursive Method:", recursive_result)
```

EXECUTION:

A screenshot of a Python IDE window titled 'AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code is displayed in a light blue background with syntax highlighting. The code defines two functions: 'factorial_iterative' and 'factorial_recursive'. It then sets 'n = 5', calculates the factorial using both methods, and prints the results. The code is as follows:

```
# Factorial using Iterative Method

def factorial_iterative(n):
    result = 1

    for i in range(1, n + 1):
        result *= i

    return result

# Factorial using Recursive Method

def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial_recursive(n - 1)

# Sample Input
n = 5

# Calculate factorial
iterative_result = factorial_iterative(n)
recursive_result = factorial_recursive(n)

# Display Output
print("Factorial using Iterative Method:", iterative_result)
print("Factorial using Recursive Method:", recursive_result)
```

OUTPUT:

FIBONACCI SERIES – ITERATIVE VS RECURSIVE

PYTHON CODE:

Fibonacci Series - Iterative Method

```
def fibonacci_iterative(n):
```

```
    series = []
```

```
    a, b = 0, 1
```

```
    for i in range(n):
```

```
        series.append(a)
```

```
        a, b = b, a + b
```

```
    return series
```

Fibonacci Series - Recursive Method

```
def fibonacci_recursive(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

```
def fibonacci_recursive_series(n):
```

```
    series = []
```

```
    for i in range(n):
```

```
        series.append(fibonacci_recursive(i))
```

```
    return series
```

```
# Sample Input
```

```
n = 6
```

```
# Generate Fibonacci series
```

```
iterative_result = fibonacci_iterative(n)
```

```
recursive_result = fibonacci_recursive_series(n)
```

```
# Display Output
```

```
print("Fibonacci Series using Iterative Method:", iterative_result)
```

```
print("Fibonacci Series using Recursive Method:", recursive_result)
```

EXECUTION:


```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help

def fibonacci_iterative(n):
    series = []

    a, b = 0, 1

    for i in range(n):
        series.append(a)
        a, b = b, a + b

    return series

# Fibonacci Series - Recursive Method

def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

def fibonacci_recursive_series(n):
    series = []

    for i in range(n):
        series.append(fibonacci_recursive(i))

    return series

# Sample Input
n = 6

# Generate Fibonacci series
iterative_result = fibonacci_iterative(n)
recursive_result = fibonacci_recursive_series(n)

# Display Output
print("Fibonacci Series using Iterative Method:", iterative_result)
print("Fibonacci Series using Recursive Method:", recursive_result)
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Fibonacci Series using Iterative Method: [0, 1, 1, 2, 3, 5]
Fibonacci Series using Recursive Method: [0, 1, 1, 2, 3, 5]
>>> |
```

MATRIX MULTIPLICATION

PYTHON CODE:

Matrix Multiplication

```
def matrix_multiply(A, B):
```

```
    rows_A = len(A)
```

```
    cols_A = len(A[0])
```

```
    rows_B = len(B)
```

```
    cols_B = len(B[0])
```

```
    # Check if matrices can be multiplied
```

```
    if cols_A != rows_B:
```

```
        return None
```

```
    # Create result matrix filled with zeros
```

```
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
```

```
    # Matrix multiplication using nested loops
```

```
    for i in range(rows_A):
```

```
        for j in range(cols_B):
```

```
            for k in range(cols_A):
```

```
                result[i][j] += A[i][k] * B[k][j]
```

```
    return result
```

```
# Sample Input
```

```
A = [[1, 2],
```

```
      [3, 4]]
```

```
B = [[5, 6],  
     [7, 8]]
```

```
# Perform Matrix Multiplication
```

```
result = matrix_multiply(A, B)
```

```
# Display Output
```

```
print("Result of Matrix Multiplication:")
```

```
for row in result:
```

```
    print(row)
```

EXECUTION:

```
AA.py - C:/Users/Agilan/OneDrive/Documents/AA.py (3.14.2)
File Edit Format Run Options Window Help

# Matrix Multiplication

def matrix_multiply(A, B):
    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    # Check if matrices can be multiplied
    if cols_A != rows_B:
        return None

    # Create result matrix filled with zeros
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    # Matrix multiplication using nested loops
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i][j] += A[i][k] * B[k][j]

    return result

# Sample Input
A = [[1, 2],
      [3, 4]]

B = [[5, 6],
      [7, 8]]

# Perform Matrix Multiplication
result = matrix_multiply(A, B)

# Display Output
print("Result of Matrix Multiplication:")
for row in result:
    print(row)
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/AA.py =====
Result of Matrix Multiplication:
[19, 22]
[43, 50]
>>> |
```

MERGE SORT

PYTHON CODE:

Merge Sort

```
def merge_sort(arr):
```

```
    # Base condition
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    # Find the middle index
```

```
    mid = len(arr) // 2
```

```
    # Divide the array into two halves
```

```
    left = merge_sort(arr[:mid])
```

```
    right = merge_sort(arr[mid:])
```

```
    # Merge the sorted halves
```

```
    return merge(left, right)
```

```
def merge(left, right):
```

```
    result = []
```

```
    i = 0
```

```
    j = 0
```

```
    # Compare elements from both arrays
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] <= right[j]:
```

```
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Sample Input
arr = [38, 27, 43, 3, 9, 82, 10]

# Perform Merge Sort
sorted_arr = merge_sort(arr)

# Display Output
print("Original Array:", arr)
print("Sorted Array:", sorted_arr)
```

EXECUTION:

```
BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)
File Edit Format Run Options Window Help

mid = len(arr) // 2

# Divide the array into two halves
left = merge_sort(arr[:mid])
right = merge_sort(arr[mid:])

# Merge the sorted halves
return merge(left, right)

def merge(left, right):
    result = []
    i = 0
    j = 0

    # Compare elements from both arrays
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Add remaining elements
    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Sample Input
arr = [38, 27, 43, 3, 9, 82, 10]

# Perform Merge Sort
sorted_arr = merge_sort(arr)

# Display Output
print("Original Array:", arr)
print("Sorted Array:", sorted_arr)
```

Ln: 49 Col: 0

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Original Array: [38, 27, 43, 3, 9, 82, 10]
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
>>>
```

QUICK SORT

PYTHON CODE:

Quick Sort

```
def quick_sort(arr):
```

```
    # Base condition
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    # Select the last element as pivot
```

```
    pivot = arr[-1]
```

```
    # Elements smaller than or equal to pivot
```

```
    left = [x for x in arr[:-1] if x <= pivot]
```

```
    # Elements greater than pivot
```

```
    right = [x for x in arr[:-1] if x > pivot]
```

```
    # Recursively sort and combine
```

```
    return quick_sort(left) + [pivot] + quick_sort(right)
```

```
# Sample Input
```

```
arr = [10, 7, 8, 9, 1, 5]
```

```
# Perform Quick Sort
```

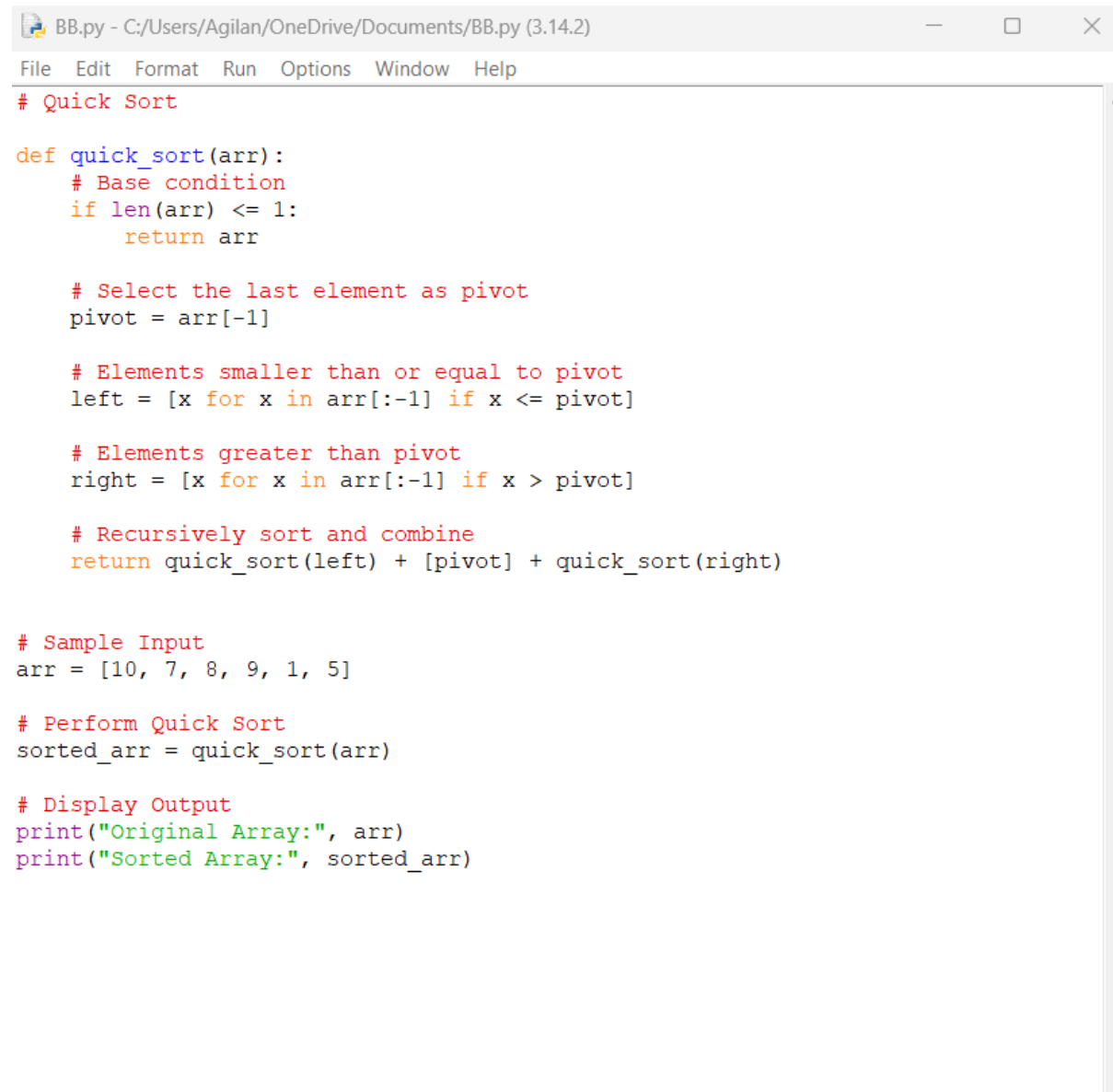
```
sorted_arr = quick_sort(arr)
```


Display Output

```
print("Original Array:", arr)
```

```
print("Sorted Array:", sorted_arr)
```

EXECUTION:

A screenshot of a Python IDE window titled "BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code editor displays a Python script for Quick Sort. The script includes a function definition for quick_sort, sample input, and print statements for output. The code is color-coded: comments are in red, function names and variables in blue, and keywords and literals in orange or green. The script defines a recursive quick_sort function that selects the last element as a pivot, partitions the array into elements less than or equal to the pivot and elements greater than the pivot, and then recursively sorts the sub-arrays. It uses a sample input array [10, 7, 8, 9, 1, 5] and prints the original and sorted arrays.

```
# Quick Sort

def quick_sort(arr):
    # Base condition
    if len(arr) <= 1:
        return arr

    # Select the last element as pivot
    pivot = arr[-1]

    # Elements smaller than or equal to pivot
    left = [x for x in arr[:-1] if x <= pivot]

    # Elements greater than pivot
    right = [x for x in arr[:-1] if x > pivot]

    # Recursively sort and combine
    return quick_sort(left) + [pivot] + quick_sort(right)

# Sample Input
arr = [10, 7, 8, 9, 1, 5]

# Perform Quick Sort
sorted_arr = quick_sort(arr)

# Display Output
print("Original Array:", arr)
print("Sorted Array:", sorted_arr)
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Original Array: [10, 7, 8, 9, 1, 5]
Sorted Array: [1, 5, 7, 8, 9, 10]
>>>
```

TOWER OF HANOI

PYTHON CODE:

Tower of Hanoi

```
def tower_of_hanoi(n, source, auxiliary, destination):
```

```
    # Base condition
```

```
    if n == 1:
```

```
        print(f'Move disk 1 from {source} to {destination}')
```

```
        return
```

```
    # Move n-1 disks from source to auxiliary
```

```
    tower_of_hanoi(n - 1, source, destination, auxiliary)
```

```
    # Move the largest disk to destination
```

```
    print(f'Move disk {n} from {source} to {destination}')
```

```
    # Move n-1 disks from auxiliary to destination
```

```
    tower_of_hanoi(n - 1, auxiliary, source, destination)
```

Sample Input

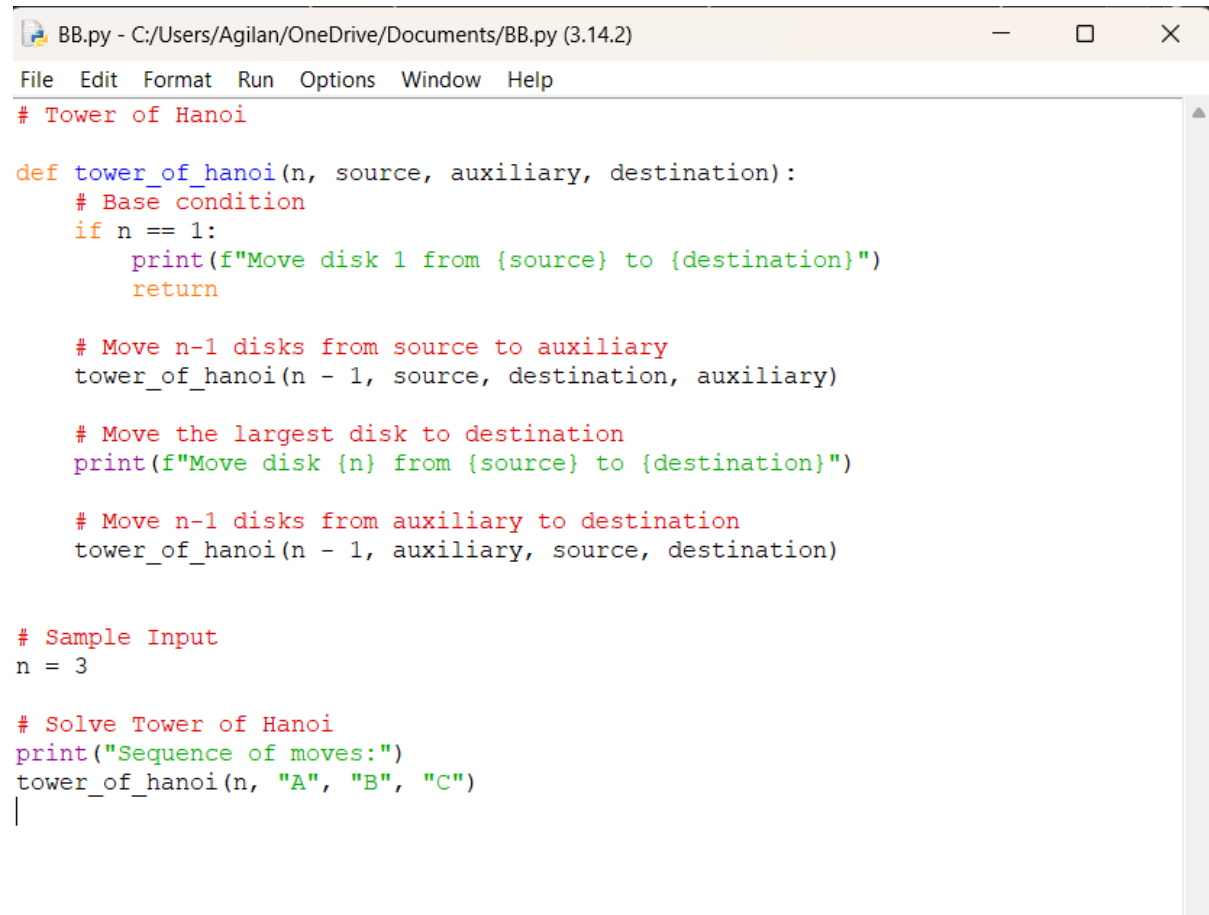
```
n = 3
```

```
# Solve Tower of Hanoi

print("Sequence of moves:")

tower_of_hanoi(n, "A", "B", "C")
```

EXECUTION:

A screenshot of a Python IDE window titled "BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code editor shows the following Python code:

```
# Tower of Hanoi

def tower_of_hanoi(n, source, auxiliary, destination):
    # Base condition
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    # Move n-1 disks from source to auxiliary
    tower_of_hanoi(n - 1, source, destination, auxiliary)

    # Move the largest disk to destination
    print(f"Move disk {n} from {source} to {destination}")

    # Move n-1 disks from auxiliary to destination
    tower_of_hanoi(n - 1, auxiliary, source, destination)

# Sample Input
n = 3

# Solve Tower of Hanoi
print("Sequence of moves:")
tower_of_hanoi(n, "A", "B", "C")
|
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Sequence of moves:
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
>>>
```

GCD – EUCLIDEAN ALGORITHM

PYTHON CODE:

GCD using Euclidean Algorithm - Recursive

```
def gcd(a, b):
```

```
    # Base condition
```

```
    if b == 0:
```

```
        return a
```

```
    # Recursive call
```

```
    return gcd(b, a % b)
```

```
# Sample Input
```

```
a = 48
```

```
b = 18
```

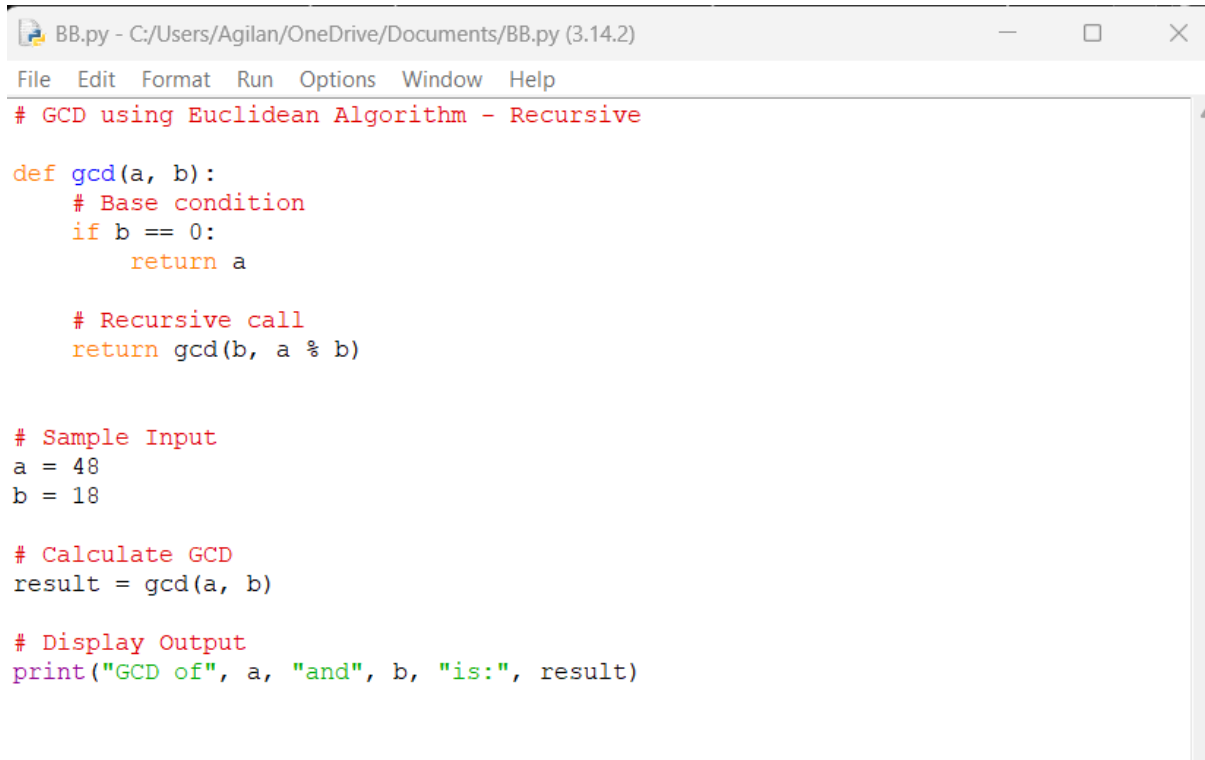
```
# Calculate GCD
```

```
result = gcd(a, b)
```

```
# Display Output
```

```
print("GCD of", a, "and", b, "is:", result)
```

EXECUTION:

A screenshot of a Python IDE window titled "BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)". The window has a menu bar with "File", "Edit", "Format", "Run", "Options", "Window", and "Help". The code is written in a light blue font on a white background. It defines a recursive function "gcd(a, b)" with a base condition "if b == 0: return a" and a recursive call "return gcd(b, a % b)". Below the function, it sets "a = 48" and "b = 18", calculates the GCD using "result = gcd(a, b)", and displays the output with "print('GCD of', a, 'and', b, 'is:', result)".

```
BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)
File Edit Format Run Options Window Help
# GCD using Euclidean Algorithm - Recursive

def gcd(a, b):
    # Base condition
    if b == 0:
        return a

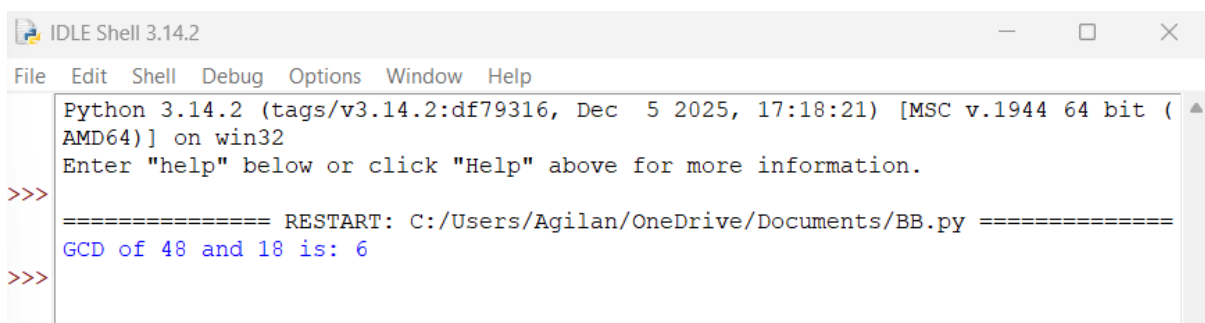
    # Recursive call
    return gcd(b, a % b)

# Sample Input
a = 48
b = 18

# Calculate GCD
result = gcd(a, b)

# Display Output
print("GCD of", a, "and", b, "is:", result)
```

OUTPUT:

A screenshot of an IDLE Shell window titled "IDLE Shell 3.14.2". The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The output is displayed in a light blue font on a white background. It shows the Python version and architecture, followed by a prompt to enter "help". Then, it shows a restart message and the output of the GCD calculation: "GCD of 48 and 18 is: 6".

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
GCD of 48 and 18 is: 6
>>>
```

INSERTION SORT

PYTHON CODE:

```
# Insertion Sort
```

```
def insertion_sort(arr):
```

```
for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    # Move elements greater than key one position ahead
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    # Insert key at the correct position
    arr[j + 1] = key

return arr
```

```
# Sample Input
```

```
arr = [12, 11, 13, 5, 6]
```

```
# Perform Insertion Sort
```

```
sorted_arr = insertion_sort(arr)
```

```
# Display Output
```

```
print("Original Array: [12, 11, 13, 5, 6]")
```

```
print("Sorted Array:", sorted_arr)
```

EXECUTION:

```
BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)
File Edit Format Run Options Window Help
# Insertion Sort

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Move elements greater than key one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        # Insert key at the correct position
        arr[j + 1] = key

    return arr

# Sample Input
arr = [12, 11, 13, 5, 6]

# Perform Insertion Sort
sorted_arr = insertion_sort(arr)

# Display Output
print("Original Array: [12, 11, 13, 5, 6]")
print("Sorted Array:", sorted_arr)
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Original Array: [12, 11, 13, 5, 6]
Sorted Array: [5, 6, 11, 12, 13]
>>>
```

HEAP SORT

PYTHON CODE:

Heap Sort

```
def heapify(arr, n, i):
```

```
    largest = i
```

```
left = 2 * i + 1
```

```
right = 2 * i + 2
```

```
# Check if left child is larger
```

```
if left < n and arr[left] > arr[largest]:
```

```
    largest = left
```

```
# Check if right child is larger
```

```
if right < n and arr[right] > arr[largest]:
```

```
    largest = right
```

```
# If largest is not the root
```

```
if largest != i:
```

```
    arr[i], arr[largest] = arr[largest], arr[i]
```

```
# Recursively heapify the affected subtree
```

```
    heapify(arr, n, largest)
```

```
def heap_sort(arr):
```

```
    n = len(arr)
```

```
# Build a max heap
```

```
for i in range(n // 2 - 1, -1, -1):
```

```
    heapify(arr, n, i)
```

```
# Extract elements one by one
```

```
for i in range(n - 1, 0, -1):
```

```
    arr[0], arr[i] = arr[i], arr[0]
```



```
# Heapify the reduced heap
```

```
heapify(arr, i, 0)
```

```
return arr
```

```
# Sample Input
```

```
arr = [4, 10, 3, 5, 1]
```

```
# Perform Heap Sort
```

```
sorted_arr = heap_sort(arr)
```

```
# Display Output
```

```
print("Original Array: [4, 10, 3, 5, 1]")
```

```
print("Sorted Array:", sorted_arr)
```

EXECUTION:

```
BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)
File Edit Format Run Options Window Help

    largest = left

    # Check if right child is larger
    if right < n and arr[right] > arr[largest]:
        largest = right

    # If largest is not the root
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]

        # Recursively heapify the affected subtree
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    # Build a max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements one by one
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]

        # Heapify the reduced heap
        heapify(arr, i, 0)

    return arr

# Sample Input
arr = [4, 10, 3, 5, 1]

# Perform Heap Sort
sorted_arr = heap_sort(arr)

# Display Output
print("Original Array: [4, 10, 3, 5, 1]")
print("Sorted Array:", sorted_arr)
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Original Array: [4, 10, 3, 5, 1]
Sorted Array: [1, 3, 4, 5, 10]
>>>
```

COUNTING INVERSIONS

PYTHON CODE:

Counting Inversions using Merge Sort

```
def merge_and_count(arr, left, mid, right):
```

```
    temp = []
```

```
    i = left
```

```
    j = mid + 1
```

```
    inversions = 0
```

```
    # Compare elements from both halves
```

```
    while i <= mid and j <= right:
```

```
        if arr[i] <= arr[j]:
```

```
            temp.append(arr[i])
```

```
            i += 1
```

```
        else:
```

```
            temp.append(arr[j])
```

```
            inversions += (mid - i + 1)
```

```
            j += 1
```

```
    # Add remaining elements
```

```
    while i <= mid:
```

```
        temp.append(arr[i])
```

```
        i += 1
```

```
    while j <= right:
```

```
        temp.append(arr[j])
```

```
        j += 1
```

```
    # Copy sorted elements back
```

```
for k in range(len(temp)):
    arr[left + k] = temp[k]
```

```
return inversions
```

```
def merge_sort_and_count(arr, left, right):
```

```
    if left >= right:
```

```
        return 0
```

```
    mid = (left + right) // 2
```

```
    inversions = 0
```

```
    # Count inversions in left half
```

```
    inversions += merge_sort_and_count(arr, left, mid)
```

```
    # Count inversions in right half
```

```
    inversions += merge_sort_and_count(arr, mid + 1, right)
```

```
    # Count split inversions
```

```
    inversions += merge_and_count(arr, left, mid, right)
```

```
    return inversions
```

```
# Sample Input
```

```
arr = [2, 4, 1, 3, 5]
```

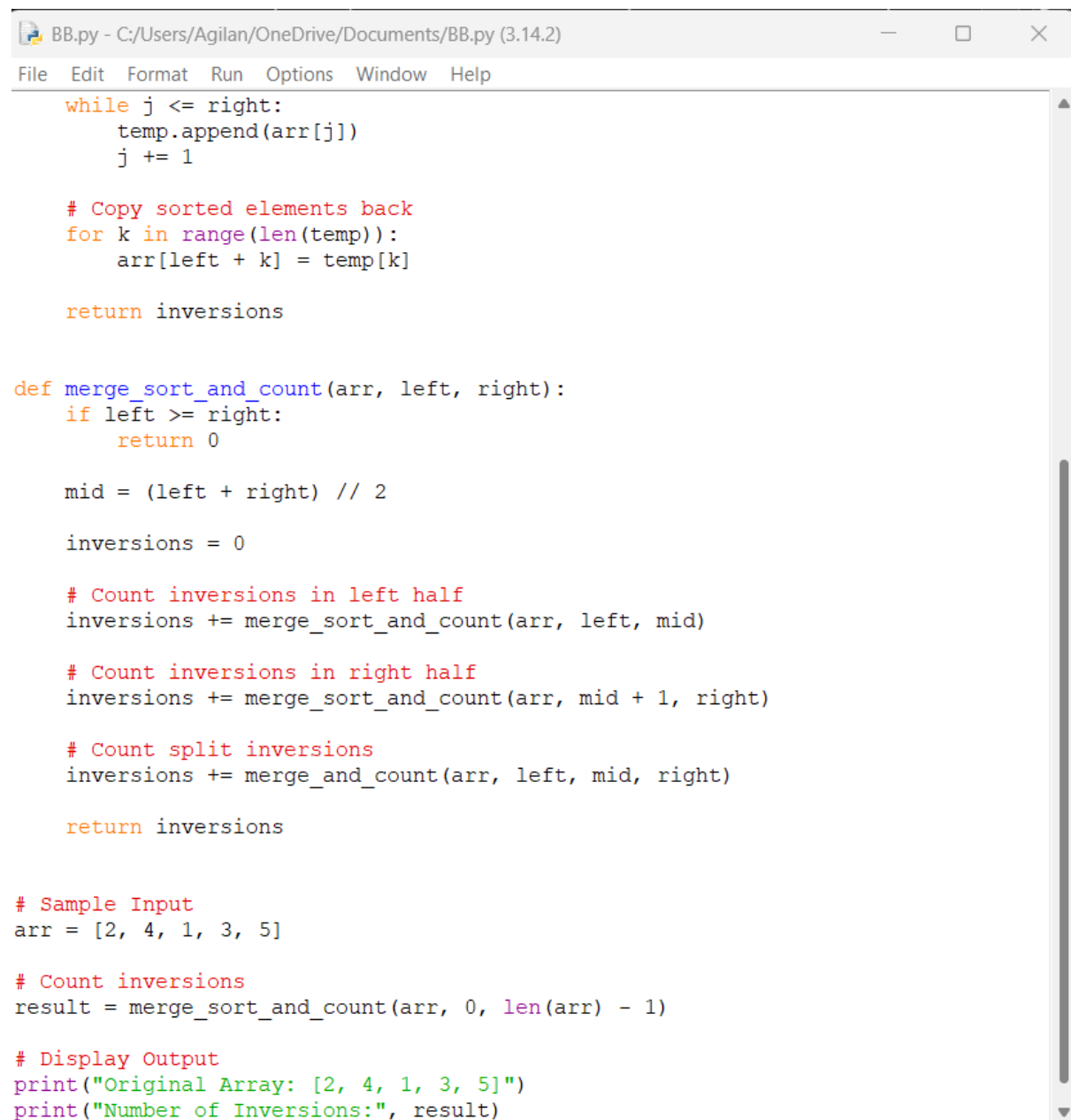
```
# Count inversions

result = merge_sort_and_count(arr, 0, len(arr) - 1)
```

```
# Display Output

print("Original Array: [2, 4, 1, 3, 5]")
print("Number of Inversions:", result)
```

EXECUTION:

A screenshot of a Python IDE window titled "BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)". The window contains Python code for a merge sort algorithm that counts inversions. The code is as follows:

```
while j <= right:
    temp.append(arr[j])
    j += 1

# Copy sorted elements back
for k in range(len(temp)):
    arr[left + k] = temp[k]

return inversions

def merge_sort_and_count(arr, left, right):
    if left >= right:
        return 0

    mid = (left + right) // 2

    inversions = 0

    # Count inversions in left half
    inversions += merge_sort_and_count(arr, left, mid)

    # Count inversions in right half
    inversions += merge_sort_and_count(arr, mid + 1, right)

    # Count split inversions
    inversions += merge_and_count(arr, left, mid, right)

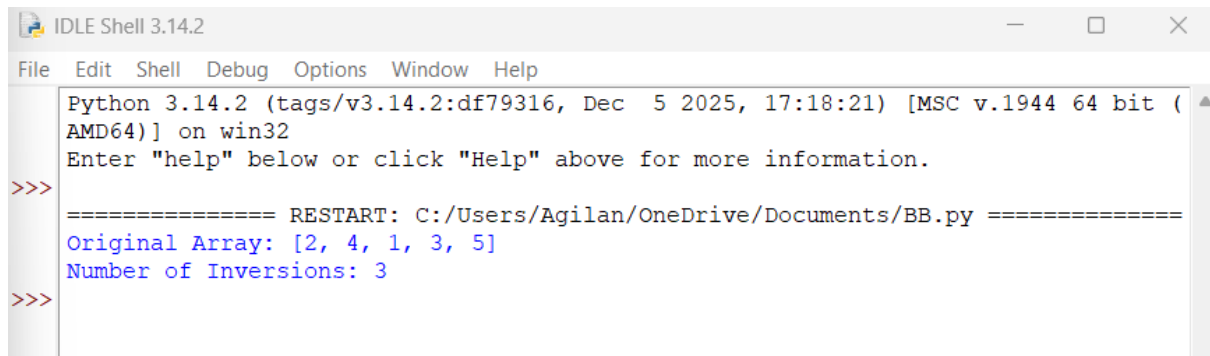
    return inversions

# Sample Input
arr = [2, 4, 1, 3, 5]

# Count inversions
result = merge_sort_and_count(arr, 0, len(arr) - 1)

# Display Output
print("Original Array: [2, 4, 1, 3, 5]")
print("Number of Inversions:", result)
```

OUTPUT:



```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Original Array: [2, 4, 1, 3, 5]
Number of Inversions: 3
>>>
```

MAXIMUM SUBARRAY SUM

PYTHON CODE:

Experiment 13: Maximum Subarray Sum

Kadane's Algorithm

def kadane(arr):

 max_current = arr[0]

 max_global = arr[0]

 for i in range(1, len(arr)):

 max_current = max(arr[i], max_current + arr[i])

 max_global = max(max_global, max_current)

 return max_global

Divide and Conquer Method

def max_crossing_sum(arr, left, mid, right):

 # Find maximum sum on the left side

 left_sum = float('-inf')

 total = 0

```
for i in range(mid, left - 1, -1):  
    total += arr[i]  
    left_sum = max(left_sum, total)  
  
# Find maximum sum on the right side  
right_sum = float('-inf')  
total = 0
```

```
for i in range(mid + 1, right + 1):  
    total += arr[i]  
    right_sum = max(right_sum, total)  
  
return left_sum + right_sum
```

```
def divide_and_conquer(arr, left, right):  
    if left == right:  
        return arr[left]  
  
    mid = (left + right) // 2  
  
    left_sum = divide_and_conquer(arr, left, mid)  
    right_sum = divide_and_conquer(arr, mid + 1, right)  
    crossing_sum = max_crossing_sum(arr, left, mid, right)  
  
    return max(left_sum, right_sum, crossing_sum)
```

```
# Sample Input
```

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]
```

```
# Calculate maximum subarray sum
```

```
kadane_result = kadane(arr)
```

```
divide_result = divide_and_conquer(arr, 0, len(arr) - 1)
```

```
# Display Output
```

```
print("Array:", arr)
```

```
print("Maximum Subarray Sum using Kadane's Algorithm:", kadane_result)
```

```
print("Maximum Subarray Sum using Divide and Conquer:", divide_result)
```

EXECUTION:


```
BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)
File Edit Format Run Options Window Help

for i in range(mid, left - 1, -1):
    total += arr[i]
    left_sum = max(left_sum, total)

# Find maximum sum on the right side
right_sum = float('-inf')
total = 0

for i in range(mid + 1, right + 1):
    total += arr[i]
    right_sum = max(right_sum, total)

return left_sum + right_sum

def divide_and_conquer(arr, left, right):
    if left == right:
        return arr[left]

    mid = (left + right) // 2

    left_sum = divide_and_conquer(arr, left, mid)
    right_sum = divide_and_conquer(arr, mid + 1, right)
    crossing_sum = max_crossing_sum(arr, left, mid, right)

    return max(left_sum, right_sum, crossing_sum)

# Sample Input
arr = [-2, -3, 4, -1, -2, 1, 5, -3]

# Calculate maximum subarray sum
kadane_result = kadane(arr)
divide_result = divide_and_conquer(arr, 0, len(arr) - 1)

# Display Output
print("Array:", arr)
print("Maximum Subarray Sum using Kadane's Algorithm:", kadane_result)
print("Maximum Subarray Sum using Divide and Conquer:", divide_result)

Ln: 59 Col: 70
```

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help

Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Array: [-2, -3, 4, -1, -2, 1, 5, -3]
Maximum Subarray Sum using Kadane's Algorithm: 7
Maximum Subarray Sum using Divide and Conquer: 7
>>>
```

EXPONENTION – ITERATIVE VS RECURSIVE FAST POWER

PYTHON CODE:

Experiment 14: Exponentiation

Iterative Method

```
def power_iterative(x, n):
```

```
    result = 1
```

```
    for i in range(n):
```

```
        result *= x
```

```
    return result
```

Recursive Fast Power Method

```
def fast_power(x, n):
```

```
    # Base condition
```

```
    if n == 0:
```

```
        return 1
```

```
    # Calculate half power
```

```
    half = fast_power(x, n // 2)
```

```
    # If exponent is even
```

```
    if n % 2 == 0:
```

```
        return half * half
```

```
    # If exponent is odd
```

```
    else:
```

```
        return x * half * half
```

```
# Sample Input
```

```
x = 2
```

```
n = 10
```

```
# Calculate power
```

```
iterative_result = power_iterative(x, n)
```

```
recursive_result = fast_power(x, n)
```

```
# Display Output
```

```
print("Base:", x)
```

```
print("Exponent:", n)
```

```
print("Result using Iterative Method:", iterative_result)
```

```
print("Result using Recursive Fast Power:", recursive_result)
```

EXECUTION:

```
BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)
File Edit Format Run Options Window Help
def power_iterative(x, n):
    result = 1

    for i in range(n):
        result *= x

    return result

# Recursive Fast Power Method
def fast_power(x, n):
    # Base condition
    if n == 0:
        return 1

    # Calculate half power
    half = fast_power(x, n // 2)

    # If exponent is even
    if n % 2 == 0:
        return half * half

    # If exponent is odd
    else:
        return x * half * half

# Sample Input
x = 2
n = 10

# Calculate power
iterative_result = power_iterative(x, n)
recursive_result = fast_power(x, n)

# Display Output
print("Base:", x)
print("Exponent:", n)
print("Result using Iterative Method:", iterative_result)
print("Result using Recursive Fast Power:", recursive_result)
```

Ln: 43 Col: 61

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Base: 2
Exponent: 10
Result using Iterative Method: 1024
Result using Recursive Fast Power: 1024
>>>
```

CLOSEST PAIR OF POINTS

PYTHON CODE:

```
import math
```

```
# Calculate Euclidean distance between two points
```

```
def distance(p1, p2):
```

```
    return math.sqrt((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2)
```

```
# Brute Force Method
```

```
def closest_pair_brute_force(points):
```

```
    min_distance = float('inf')
```

```
    closest_points = None
```

```
    n = len(points)
```

```
    for i in range(n):
```

```
        for j in range(i + 1, n):
```

```
            d = distance(points[i], points[j])
```

```
            if d < min_distance:
```

```
                min_distance = d
```

```
                closest_points = (points[i], points[j])
```

```
    return closest_points, min_distance
```

```
# Divide and Conquer Method
```

```

def closest_pair_divide_conquer(points):
    points_sorted_x = sorted(points, key=lambda p: p[0])
    points_sorted_y = sorted(points, key=lambda p: p[1])

    return closest_pair_recursive(points_sorted_x, points_sorted_y)

def closest_pair_recursive(points_x, points_y):
    n = len(points_x)

    # For small number of points, use brute force
    if n <= 3:
        return closest_pair_brute_force(points_x)

    mid = n // 2
    left_x = points_x[:mid]
    right_x = points_x[mid:]

    mid_x = points_x[mid][0]

    left_y = [p for p in points_y if p[0] <= mid_x]
    right_y = [p for p in points_y if p[0] > mid_x]

    # Find closest pair in left and right halves
    left_pair, left_distance = closest_pair_recursive(left_x, left_y)
    right_pair, right_distance = closest_pair_recursive(right_x, right_y)

    # Choose the smaller distance
    if left_distance < right_distance:

```

```
    min_distance = left_distance
    closest_points = left_pair
else:
    min_distance = right_distance
    closest_points = right_pair

# Check points near the dividing line
strip = [
    p for p in points_y
    if abs(p[0] - mid_x) < min_distance
]
```

```
# Compare points in the strip
for i in range(len(strip)):
    for j in range(i + 1, min(i + 7, len(strip))):
        d = distance(strip[i], strip[j])

        if d < min_distance:
            min_distance = d
            closest_points = (strip[i], strip[j])
```

```
return closest_points, min_distance
```

```
# Sample Input
```

```
points = [(1, 2), (4, 5), (7, 8), (3, 1)]
```

```
# Brute Force
```

```
brute_pair, brute_distance = closest_pair_brute_force(points)
```

```
# Divide and Conquer
```

```
divide_pair, divide_distance = closest_pair_divide_conquer(points)
```

```
# Display Output
```

```
print("Points:", points)
```

```
print("\nBrute Force Method:")
```

```
print("Closest Pair:", brute_pair)
```

```
print("Distance:", math.sqrt(5))
```

```
print("\nDivide and Conquer Method:")
```

```
print("Closest Pair:", divide_pair)
```

```
print("Distance:", divide_distance)
```

EXECUTION:


```
BB.py - C:/Users/Agilan/OneDrive/Documents/BB.py (3.14.2)
File Edit Format Run Options Window Help

    min_distance = right_distance
    closest_points = right_pair

    # Check points near the dividing line
    strip = [
        p for p in points_y
        if abs(p[0] - mid_x) < min_distance
    ]

    # Compare points in the strip
    for i in range(len(strip)):
        for j in range(i + 1, min(i + 7, len(strip))):
            d = distance(strip[i], strip[j])

            if d < min_distance:
                min_distance = d
                closest_points = (strip[i], strip[j])

    return closest_points, min_distance

# Sample Input
points = [(1, 2), (4, 5), (7, 8), (3, 1)]

# Brute Force
brute_pair, brute_distance = closest_pair_brute_force(points)

# Divide and Conquer
divide_pair, divide_distance = closest_pair_divide_conquer(points)

# Display Output
print("Points:", points)

print("\nBrute Force Method:")
print("Closest Pair:", brute_pair)
print("Distance:", math.sqrt(5))

print("\nDivide and Conquer Method:")
print("Closest Pair:", divide_pair)
print("Distance:", divide_distance)
```

Ln: 99 Col: 35

OUTPUT:

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Agilan/OneDrive/Documents/BB.py =====
Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Brute Force Method:
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979

Divide and Conquer Method:
Closest Pair: ((1, 2), (3, 1))
Distance: 2.23606797749979
>>>
```